

FatTail Labs — IKI Product System Design v0.4

Status: DRAFT. Not BUILD AUTHORITY, not a plan. High-level design against `DRAFT-FatTail-Labs-IKI-Product-System-Spec-v0_2.md`.

Changes in v0.4: many streams, not one (§1b). Coach: the next evolution carries other streams Massive supplies — futures, crypto, economic — and this was part of the original vision, not an extension of it. The consequence is structural: the stream becomes a stream *of a kind*, a part declares which kind it needs, and **each kind needs its own archive started before the product that will want it.**

Changes in v0.3: the level the first two versions missed. **One stream, many consumers** (§1a) — every tool derives from one common data stream, the source is pluggable and the parts never know which one they are reading. **Parts have faces** (§2.2), not one presentation: a desk face and an Analyzer overlay are the same product shown two ways, which is why owning GEX makes it appear in both. **Capabilities are a second product class** (§2.3) — Time Machine is one entitlement across every surface, not a per-part unlock. **Surfaces are unbounded** (§4.3), including unique ones like Width Fit Rank that are not variations of anything. And **Strategy Lab is the same paradigm with the arrow reversed** (§8a).

Changes in v0.2: the rudimentary build has a **job** — retaining Observers whose trial ends (spec §0a) — and that changes what v1 must contain. New §3.1 on the two entrances, §4.2 on the gift as the first two parts, and a success criterion that is retention rather than feature coverage.

Filename: provisional — Coach names it.

Altitude: the shape of the system, not the detail of any part. Enough that the pieces can be argued about and the boundaries can be checked; not enough to build from. Every open in the spec's §10 stays open here.

Scope statement (DL-539)

Active program: IKI Product System.

Files / trees this document touches: none. Design only — no code, no routes, no schema.

Touches outside program: NONE.

Advisor honesty, prominent. This design is written **without a disk read**. Nobody has looked at how the runner loads a template today, what a template's contract actually contains, how the help system attaches to a feature, or what the wiki agent expects when something declares intent. Every place

that matters is marked **[unverified]**. Where this design and the tree disagree, **the tree wins** — the pattern has already corrected this program four times this week (the envelope, the marks tape, the flat day, the launchd plist).

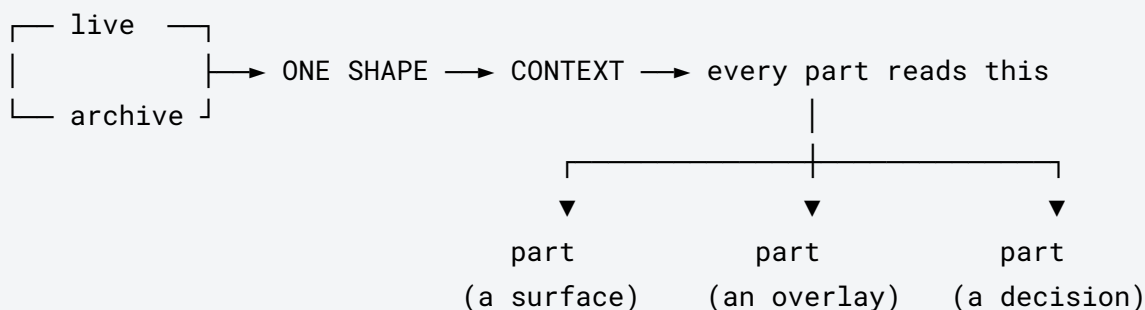
1. The shape in one paragraph

A **product** is a constellation, not a template. The **Factory** makes constellations with agents pulling work; a human promotes them. **IKI Lab** is where a member finds one, tries it in the place it will live, buys it, and arranges it on a desk. The **Runner** is an execution environment that loads products, gives each one a context, and keeps them from breaking each other. Everything a member can click is a thing that will work, because the system refuses to offer what it cannot deliver.

1a. One stream, many consumers

This is the property the whole system rests on, and it is already proven rather than hoped for.

Every tool's functionality derives from **one common stream of data**. A generation of chain arrives; every part reads that same generation. Time Machine's rebind established this in code — one context, every consumer reading from it, the live source *structurally unreachable* under a playhead. It was built to fix a defect; what it actually settled is that any number of parts can share one stream without coordinating with each other.



The source is pluggable. The shape is fixed.
The parts never know which one they are reading.

Four consequences, and they are why the catalog can scale.

A new product is not a new data path. Not a new subscription, not more load on the vendor, not another socket. It is a new *interpretation* of a generation that already arrived. Ten parts on a desk are one stream read ten ways.

Replay works for every product automatically. A part built next year replays days collected this year, without anyone building anything, because it reads the same generations. **Time Machine is not a feature each product implements — it is a property of the stream.** Which also means the archive gets more valuable as the catalog grows: same days, more ways to read them.

A second face is nearly free. GEX overlaying the tent is not a second GEX. It is the same computation against the same generation, rendered differently (§2.2).

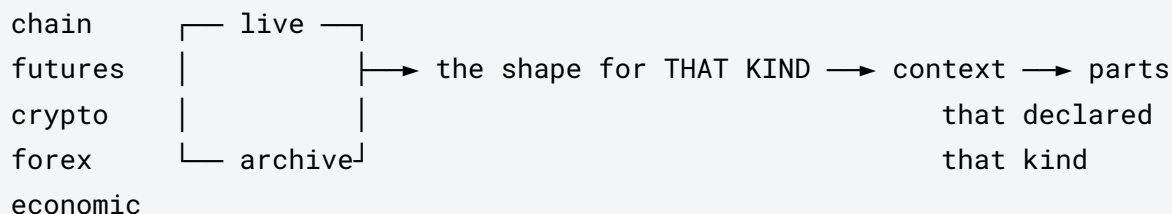
And a third source plugs in behind the same shape. A simulated day, another vendor, a synthetic session built for teaching — every existing part works against it without knowing. Nobody has to build that; it is what one-stream-many-consumers already gives you.

Coach: this was the original intent — same data, piped from a different source, one live, one archived, same surfaces. The One Source work was not a redesign; it was the architecture finally being implemented as intended. Which is why the defect it fixed looked the way it did: the price moved and nothing else did, because only one consumer had been wired to the intent.

1b. Many streams, not one

Coach: the next evolution includes other streams Massive supplies — futures products, crypto, forex, economic data. This was part of the original vision.

§1a is true and incomplete. **One stream, many consumers** describes how a single kind of data reaches many parts. What it does not yet carry is that **the stream is a stream of a kind**, and there will be several.



The substitutability that works today does not extend for free. Live and archive behind one shape works because both are chain generations. A futures tick is not strikes and greeks. An economic release is a scheduled event, not a continuous flow. **Same architecture, a shape per kind** — not one shape stretched to cover everything, which is how a data model becomes a lowest common denominator and stops being honest about any of its inputs.

1b.1 What already handles it

The contract, doing more work rather than being replaced. Today a part's *needs* implicitly means a chain generation. Made explicit, **a part declares the kind of stream it needs** and the environment satisfies it or refuses to load the part. That is the mechanism from §2.1 with the assumption removed.

Which means a member's desk can hold parts reading different kinds side by side, and each one gets what it asked for. The Runner does not need to know what futures are; it needs to know that a part asked for them and whether that kind is available.

1b.2 What it opens

Parts that read more than one kind. A butterfly heatmap with an economic calendar overlay reads two streams at different rhythms — one continuous, one a handful of scheduled events. **One context per kind, or one context holding several?** That decides whether cross-stream parts are natural or awkward, and it is a real design question this document does not answer.

Crypto and forex are close enough to share a shape, and that is worth checking rather than assuming. Both are continuous quote streams with no expiry, no chain, and — unlike equities — no session boundary. If one shape serves both, that is one archive format and one set of parts rather than two. If it does not, the difference is worth knowing before either is captured. *Advisor: check, do not assume. This platform has been bitten by an assumption that looked obviously true three times this fortnight.*

Rhythm is not uniform. Chain arrives at 2–5 seconds. An economic release arrives eight times a month. Crypto and forex run around the clock with no close at all, which means **the session boundary — the thing the trading date, the calendar and the archive's day folders are all built on — does not exist for them.** That is not a detail. Every day-shaped assumption in the archive and in Time Machine is an equity-session assumption. A clock built for one is wrong for the other, and the playhead — which is currently *the* clock — becomes the clock *of a kind* under replay.

1b.3 The consequence that cannot be deferred

Each kind needs its own archive, and capture has to start before the product does.

The tap captures chains. Futures replay requires capturing futures — from the moment capture starts, and **permanently unavailable for everything before it.**

This platform has learned that lesson three times in a fortnight: the wing band, which is fixed for every day already banked; the ODTE-only limit, same; and VIX, which sat as a proxy for thirteen days because nobody checked. Each was cheap to fix going forward and impossible to fix backward.

So the rule that falls out of it: if a stream is worth having in the future, capture starts before the product that will want it. Collecting is cheap. Not having collected is permanent. That is an argument for starting a futures or economic tap **while** the chain products are being built, rather than when someone asks for the first futures part.

Advisor: this is the one item in this document with a cost for waiting. Everything else can be designed later at the same price.

2. The object model

Six objects. Everything else is a relationship between them.

Object	What it is	Notes
--------	------------	-------

Part	One runnable product. Declares what it needs, what it produces, and what shape it requires.	§2.1
Arrangement	A set of parts in a layout. Made by someone, so it has an author and a version.	Spec §14.3 — whether this is a <i>product</i> is Coach's
Constellation	A part plus its guide, embedded help, wiki page and landing page. This is what ships.	Spec §5
Context	What a running part is given: data, clock, surfaces. One per Runner instance per stream kind (§1b).	§4
Stream kind	Chain · futures · crypto · forex · economic. Each has its own shape and its own archive.	§1b
Card	A unit of Factory work. Carries its own state in language a human reads.	§6
Capability	An environment-level entitlement that parts <i>use</i> , not a part itself. Time Machine is the first.	§2.3
Grant	What a member holds that lets a part or a capability run for them.	Spec §14.9 — commerce shape is Coach's; this design only needs <i>something</i> to exist

2.1 A part's contract

The contract is what makes a part a product rather than a function the host happens to call. Four declarations:

Declaration	Why it exists
-------------	---------------

Needs	Which stream kind (§1b), plus services — marks, the archive, the playhead. The environment satisfies it or refuses to load the part. A part that does not name its kind is assuming one, which is how the assumption got buried the first time.
Produces	What it renders, and — for the REST and pipeline paths — what it returns. Without this a part can only paint.
Shape	Fully responsive · minimum size · locked aspect ratio (spec §0.4).
Identity	Name, version, author, credentials.

The whole system leans on this. A declared contract is what lets a layout be validated before it is saved, a landing page be generated, a part be called over REST, and a demo be run without special-casing. **[unverified]** — today's templates almost certainly declare far less than this, and the gap between what they declare and what this needs is the single largest unknown in the design.

2.2 A part has faces, not one presentation

Coach: when GEX is added to the Runner, it also appears in the Analyzer. A part is not bound to one surface. Owning it means it shows up wherever it has something to say.

A face is not a resize. Responsive means the same presentation at another size. A face is a *different* presentation for a different surface:

Face	What it is
Desk	GEX as a cell in its own right — the profile, standing alone
Analyzer overlay	GEX as a layer against someone else's tent

Both are GEX. Neither is a resize of the other, and the data behind them is identical (§1a).

So the contract declares faces, and each face carries its own shape and its own surface. That replaces the single shape declaration in §2.1 and answers three things at once: what a part offers a given surface, what a layout has to satisfy, and what appears when someone owns it.

Two things this settles or opens:

- **The Analyzer is a surface, not a part.** Parts appear *on* it. That closes an ambiguity the first draft left.
- **Not every part has every face.** A regime badge may be desk-only with nothing sensible to overlay. That is fine — but a part with no Analyzer face must be *legibly* absent there rather than looking broken.
- **Whether a face appears by default or is toggled is Coach's.** *Advisor lean: toggled and remembered — an Analyzer with every owned overlay on at once is unreadable.*

2.3 Capabilities are a second class

Coach: Time Machine is the same thing across heatmaps and Analyzer. One entitlement. Buy it and the scrubber works everywhere it appears — including on parts bought later.

That makes it **not a part**. It is a capability of the environment that parts get to use, and it is the first of its kind:

	Part	Capability
Placed on a desk	yes	no
Bought once, applies everywhere	no	yes
Declared in a part's <i>needs</i>	—	yes: a part may need the playhead
Priced on	what it shows	what it makes possible

A part declaring it needs the playhead is satisfied or not depending on what the member holds.

Which means the gift heatmaps run live for someone without Time Machine, and light up their scrubber the moment it is bought — no per-part unlock, no repurchase.

Advisor on the pricing frame, which Coach stated plainly: Time Machine plus Algo alerts is not a cheap add-on. It changes what the tools are *for*. A live heatmap shows what is happening; a scrubbable one is the only way to learn to read one. An alert built against a replayed day, watched firing, is the difference between having a rule and knowing the rule works. That is a **practice environment**, and it sits exactly on capacity-over-dependency: repetition against real conditions with

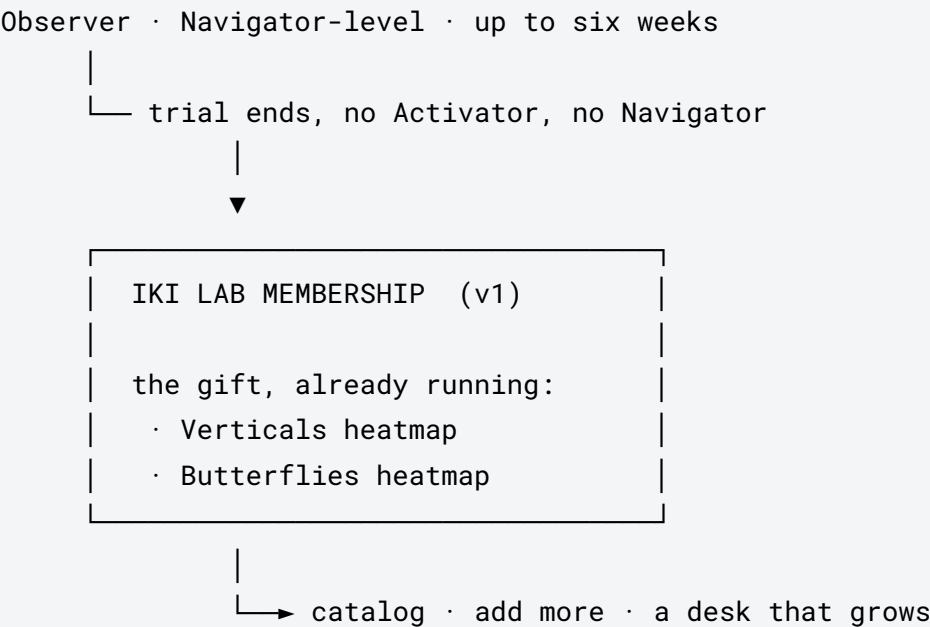
the outcome available is how a member ends up not needing the platform. It is not withheld capability — it is the training apparatus.

Advisor flag, not a finding. The gift heatmaps carry a Time Machine strip on every host. Someone holding the gift without the capability must be able to see **what they do not have** rather than wondering why nothing happens — the same discipline as the demo badge. Gating well is different from hiding.

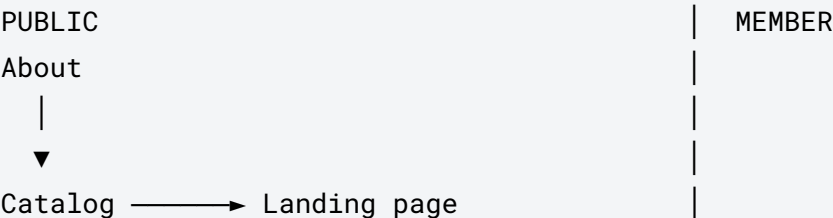
3. The member funnel

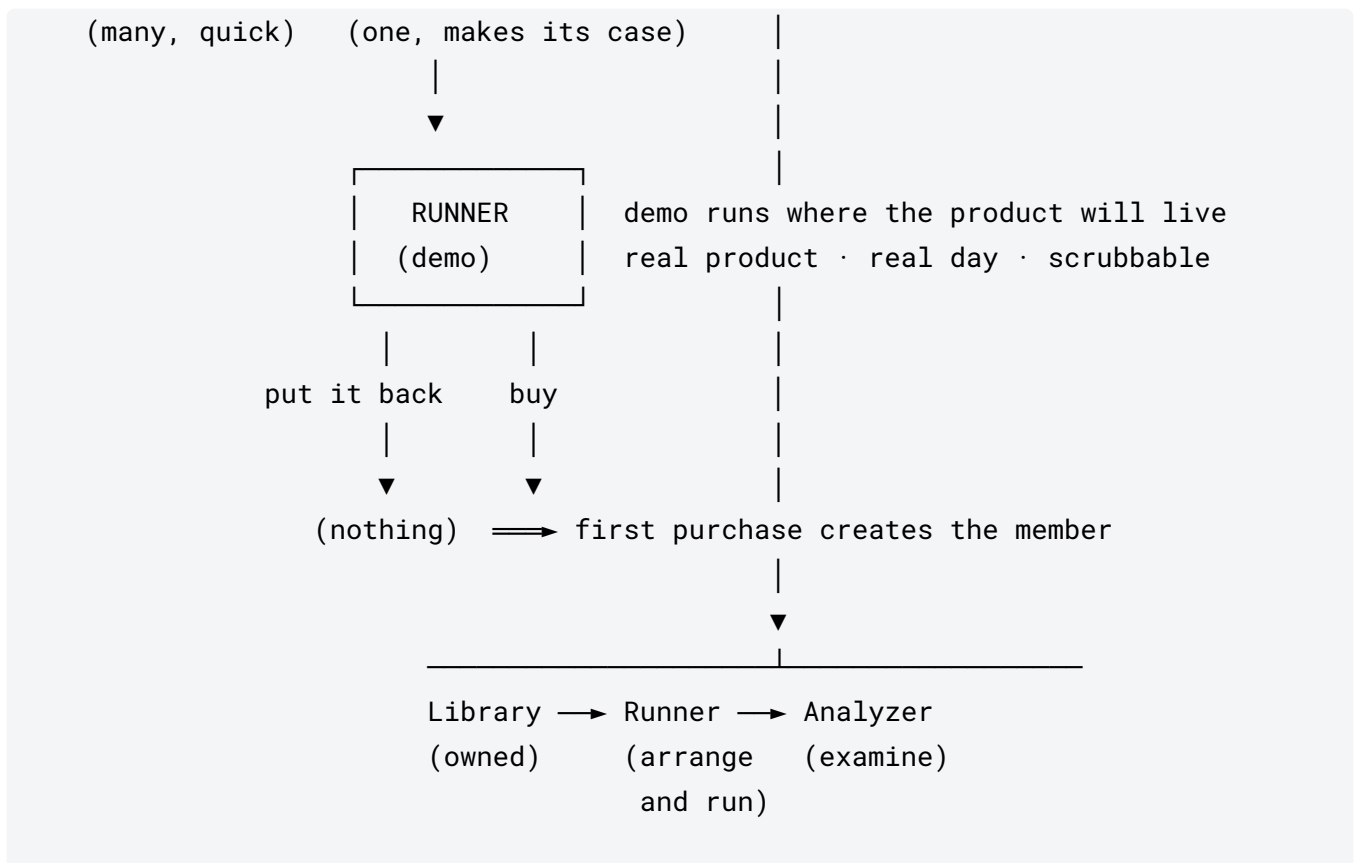
Two entrances converging on one member. Path A is what is being built now.

A · FROM A TRIAL (spec §0.21–26 – the retention job, first)



B · DIRECT (spec §0.13 – later)





Both paths end in the same place. A member with a library, a runner and an analyzer. What differs is how they arrived and what is already on the desk.

The membership boundary sits at the first purchase, not at signup. Everything left of it is public: about, catalog, landing pages, and a demo that is the real product on real data.

Three consequences of that boundary.

The demo is the trial, so there is no trial period and nothing expires. A member pays when they want to keep something on their desk.

The catalog and landing pages are public and indexable — the same shape as catalog and course pages under the existing SEO layer. Landing pages are an acquisition surface, not merely a purchase step.

And the archive serves a public path for the first time. A demo pulls an archived day into an unauthenticated browser. Every archive route currently requires a bearer, deliberately. This is spec §14.12 and it is the most consequential open in the funnel.

3.1 What path A requires that path B does not

The retention job (spec §0a) puts three constraints on v1 that a first version would not otherwise carry.

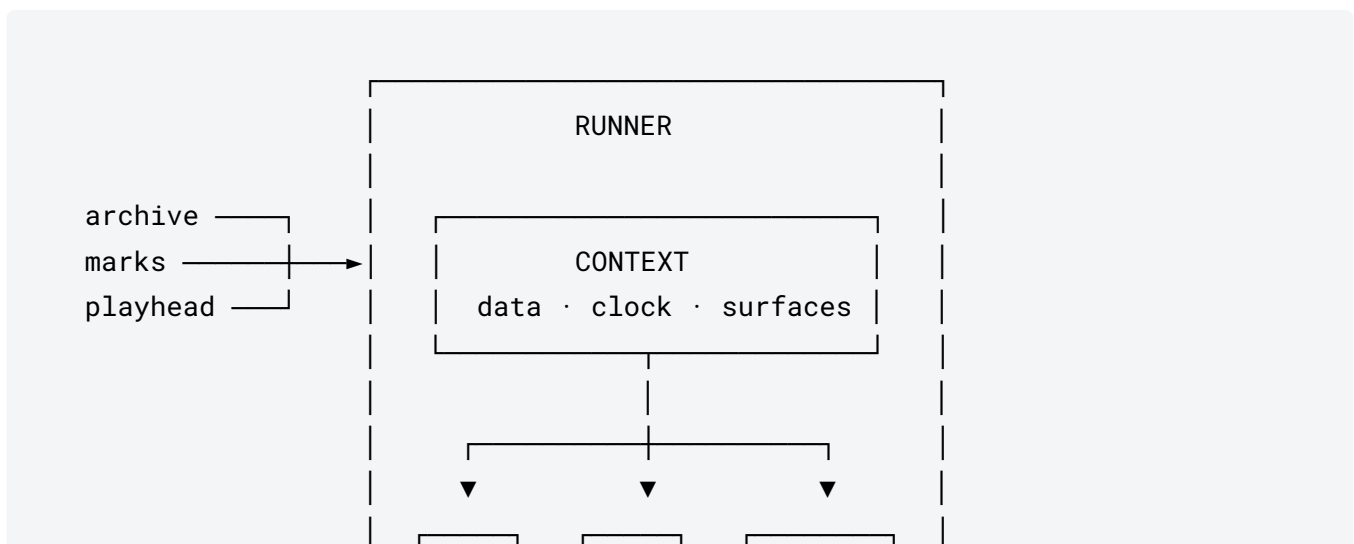
The desk is not empty. Someone who had Navigator for six weeks and lands on a blank Runner has been demoted, whatever it is called. The gift — Verticals and Butterflies — is what gives the surface a pulse before any purchase.

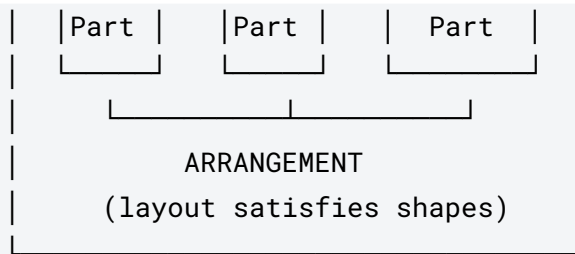
It has to be worth opening tomorrow. The success test is not *does it run templates*; it is **does a lapsed Observer open it a week later** (spec §0a.2). That is measurable and it is the only criterion that matches the job.

The transition has to be honest about what changed. Something is genuinely taken (spec §0.25). Whether their Practice history — journal, trade log, retros — survives the transition is **spec §14.13 and undirected**; the design flags it because losing one's own six weeks of work is the most memorable part of a downgrade and the thing most likely to end the relationship.

4. The Runner as an execution environment

The claim in spec §0.1 is that this is an operating system for products. That claim is only true if the environment does four things a template host does not.





Parts read only from the context. There is no second source to read.

Load and unload. A part is installed, versioned, and removable. A template that only ever exists because the host imported it cannot be bought or removed.

One context, and nothing else reachable. Every part reads the same data and the same clock. This is not a convention — it is the Time Machine pattern, where the live source is *structurally unreachable* under a playhead rather than merely unused. With several parts on one surface, that stops being tidiness and becomes the isolation rule: a part cannot reach something it was not given, so it cannot break another part by reading the wrong thing. **Spec §14.4 — Coach rules whether this is the model.**

Layout satisfies declarations. The arrangement is not pixels; it is a set of parts whose shape declarations are satisfied. That is what lets an arrangement be validated before saving, refuse rather than render something unusable, and travel between a laptop and a monitor while staying recognisably the same desk.

A failing part is a named hole, not a broken dashboard. One part that cannot load says so in its own cell. The desk does not go down.

4.1 The first two parts are the gift

Verticals and Butterflies heatmaps (spec §0.26) are v1's parts. That is a design advantage, not only a product decision:

- **The environment is built around understood products**, not hypothetical ones. Every question §2.1 asks of a contract — what it needs, produces, requires — has a real answer for these two rather than a guessed one.
- **They exercise the shape declaration honestly.** A heatmap has strike-height requirements; it is exactly the case where *fully responsive* is false and a minimum or ratio is real.
- **Two parts is enough to prove the environment and not enough to hide behind.** One part is a

template host with extra words. Two on one surface is where isolation, layout satisfaction, and a named hole for a failing part all become real.

Advisor: if the contract in §2.1 cannot be expressed for these two, it cannot be expressed at all, and that is the cheapest possible place to discover it.

4.2 Surfaces are unbounded, and two kinds of them

Coach: there are virtually unlimited surfaces that can be created — variations of familiar ones, and unique ones like **Width Fit Rank**.

Kind	Example	Where the value is
Familiar, varied	a heatmap rendered differently; a tent with different overlays	the variation
Unique	Width Fit Rank — which width best fits this book right now	it looks like nothing else, because it answers a question nobody had a picture for

The second kind is what a template engine cannot produce and an environment can. A template engine has a shape and you fill it. An environment with one stream, a declared contract, and somewhere to put things lets somebody build a surface nobody anticipated — including Coach, later.

And the economics stay flat. A unique surface costs the same as a variation, because both are one more consumer of a stream already arriving (§1a).

One constraint holds it together: a new surface is still a part with a contract. Unlimited variety, one shape of declaration. Without that the catalog fragments and nothing composes.

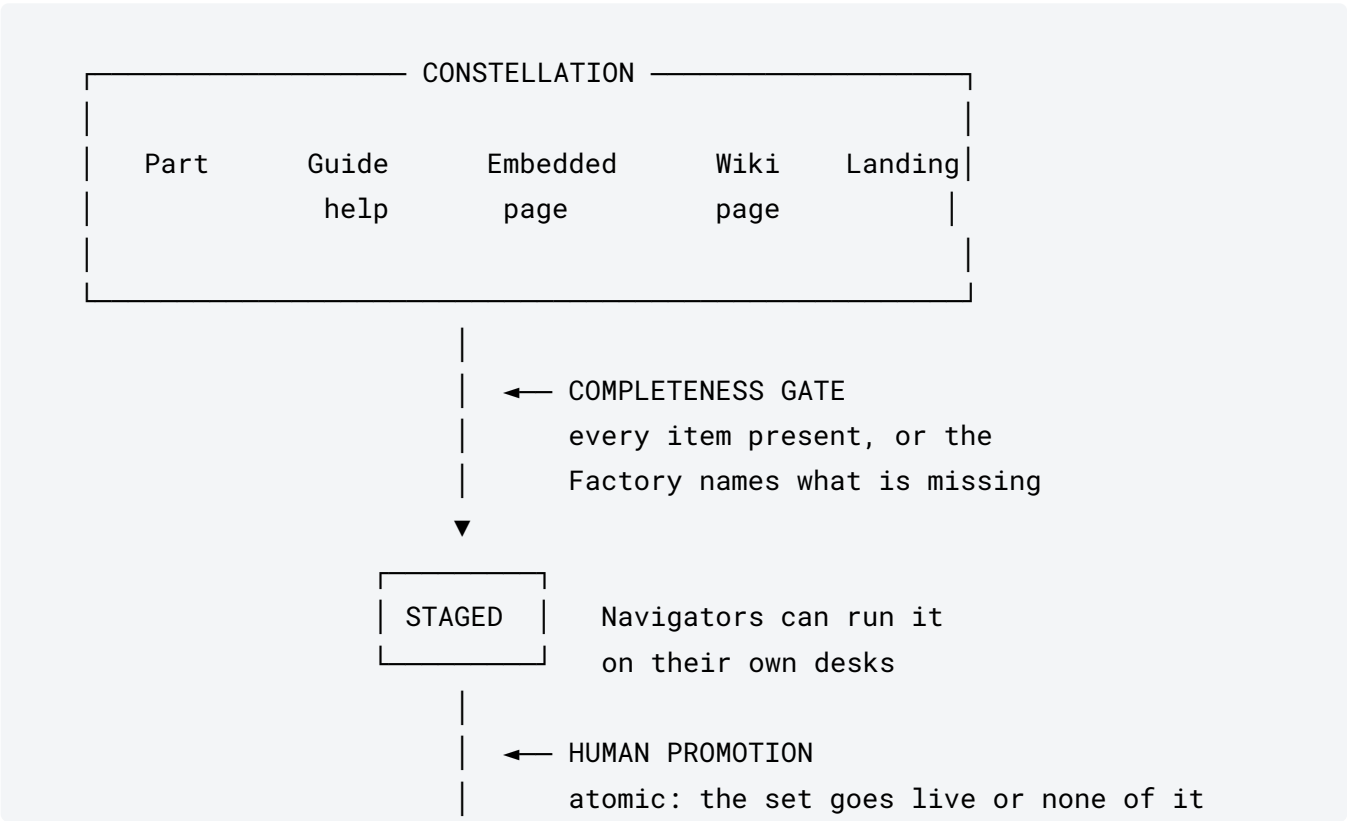
4.3 Temporary parts

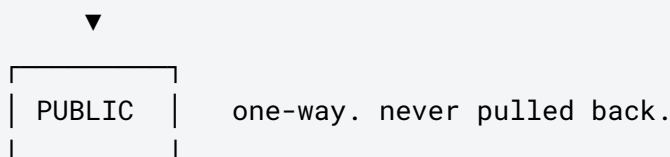
A demo part and a bought part are the same code in the same environment. What differs is that a demo is temporary, and the platform already has a settled pattern for temporary things — rehearsal objects.

Rule	
Badged	Visibly not owned. A member is never unsure which parts they have bought.
Isolated	No alerts, no trade log, no persistence.
Disposed with an announcement	<i>Put it back</i> removes it and says so. Nothing vanishes silently.
Non-destructive	A saved arrangement is restored exactly, or the demo occupies a slot that is clearly temporary.

This is the third use of that pattern — rehearsal objects, then dark positions, now demos. Worth naming as a general rule rather than rediscovering it a fourth time: *a temporary thing is badged, isolated, and disposed with an announcement.*

5. The constellation and the promotion gate





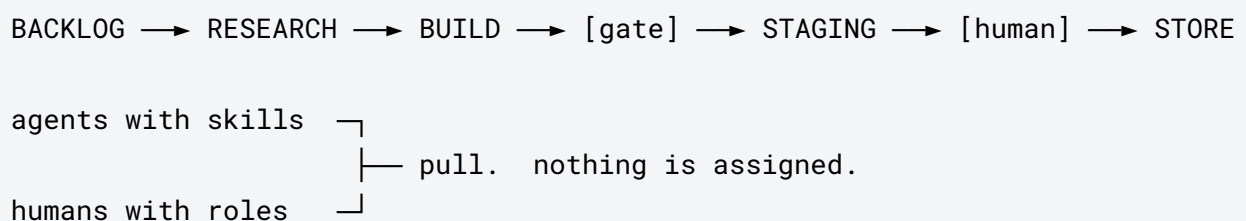
The gate is before staging, not after. Staging receives a whole constellation; it is not where one is assembled. That is spec §0.17 and it is the load-bearing sequencing decision in the Factory.

Completeness is checkable, not asserted. The Factory says a product is not stageable **and names what is missing** — the same discipline as a calendar naming the day it cannot load. This puts the wiki agent on the critical path: a product waiting on its wiki entry is a product not shipping. **[unverified]** — whether the wiki agent can be asked *is this done* is unknown and it decides whether the gate is real or aspirational.

Staging is a limited-visibility release, not a rehearsal area. Navigators run a finished thing on their own arrangements, which finds what a checklist does not.

Promotion is atomic and human. Atomic because a product live with no help, or a wiki page pointing at something unpurchasable, is a half-assembled constellation — the same shape as the one-space transaction rule. Human because agents can prove every card is closed but cannot judge whether this should exist in public under Coach's name.

6. The Factory



First incarnation: Kanban with pull. That is what exists and it works.

The relationship to Spaces, which Coach has already ruled. The Factory is the first Spaces instance,

admin-only, with the lanes dying. Those reconcile rather than collide: **Spaces replaces the lanes, not the pulling.** Pull is already entity behaviour, so the working method does not change when the substrate does.

Two disciplines that cost nothing now and keep the later swap cheap:

1. **Nothing addresses a specific agent.** A card says what it is, never who works it. Addressing surviving the death of the lanes is Coach's own lie-check for the Spaces proof.
2. **A card carries its own state in readable language,** not only in the column it sits in. If the column is the only thing that knows a card has been researched, the column cannot be removed.

And the completeness gate stops being a checklist under Spaces. A constellation is complete when every card in its set is claimed, worked and released — visible in the space rather than needing a separate list. The gate and the substrate become the same fact seen twice.

7. Delivery paths

Three ways to consume one product. **This is where the contract earns its keep** — a part that only paints cannot be called.

Path	What it needs from the contract
Desk	Needs · shape. The Runner satisfies and places it.
REST	Needs · produces. No surface; inputs in, outputs out.
Pipeline	Needs · produces · identity. Wired into something else, so version matters.

Advisor: if a part's *produces* is declared for the desk path anyway, REST is close to free and pipeline is REST with versioning discipline. If it is not declared, all three are separate implementations. **Spec §14.6 — whether all three are v1 is Coach's**, but the contract decides whether that question is expensive or cheap.

8. Boundaries

Where this system stops, so it is not designed as though on empty ground.

Boundary	
Options Lab	Provides the archive, the clock, and the existing template surface. Not changed by this system.
Time Machine	The demo's data and clock. Spec §14.10 — whether the Runner shares its playhead.
Practice	Supplies the symbol universe a product runs against.
Help system	Reused as-is (§0.15). The existing build-time obligation keeps guides from drifting.
Wiki + Oscar	Authors the wiki page from a declaration of intent. Producers do not learn the wiki's shape.
Identity / payments	Not entered. First purchase creates a member (§0.14); how is Coach's and Mike's.
SEO layer	Landing pages are public and indexable, same shape as existing catalog pages.
No-profit-claims	Applies to every landing page. Products sell on capability and process, never outcome. Tango gates it.

8a. Strategy Lab, forward compatibility

Not scope. Recorded because the paradigm is the same with the arrow reversed.

A Runner part **interprets** a stream: given this generation, show me something. A bot **acts on** one: given this generation, decide something. Same input, same clock, same replay.

Which means most of this transfers rather than being rebuilt. **The contract already has produces in it** — a bot is a part whose produces is a decision rather than a rendering. It runs against a generation, replays against a collected day, and can be versioned, sold and granted the same way.

And backtesting is honest by construction. A bot tested against the archive is tested against exactly what the desk saw — native cadence, same holes named the same way. Not a reconstruction and not a vendor's daily bars. That is the gold data plane, and it is why the ODTE boundary matters and why the VIX proxy had to be fixed.

One distinction to hold early. A part that renders is safe to run anywhere. A part that **decides** has consequences — and the platform already has the vocabulary: rehearsal objects, no writes under a playhead, a human at the promotion gate. Strategy Lab inherits all of it rather than needing its own.

9. What has to be read before this becomes a plan

The four things marked **[unverified]**, in the order they matter:

1. **What a template declares today.** The gap between that and the contract in §2.1–2.2 is the largest unknown in the design and it determines the size of the whole program. **Faces make this sharper:** if a template today has no concept of rendering itself two ways, the face model is new work rather than a declaration. **And stream kind makes it sharper again:** today every template assumes chain, so naming the kind is a change to every one of them.
2. **How the Runner loads a template now.** Whether load/unload/version is a small addition or a rewrite.
3. **Whether the wiki agent can answer *is this done*.** Decides whether the completeness gate is real or aspirational.
4. **How the help system attaches to a feature.** Decides whether per-product help is reuse or new work.

A disk read on those four turns this design from plausible into checkable. Nothing should be planned before it.

10. Open

Every open in the spec's §10 stands, untouched. This design adds none and decides none.

Three it makes sharper rather than answering:

- **§14.3 (parts and arrangements as two classes).** The design needs both objects to exist regardless; whether the arrangement is *sellable* is a commerce question, not a structural one.
 - **§14.4 (isolation model).** The one-context pattern is what makes the operating-system claim true rather than aspirational. If Coach rules otherwise, §4 changes materially.
 - **§14.13 (Practice history at the transition).** Not architecture — retention. But it is the single decision most likely to determine whether path A works, and it costs little either way.
-

11. Document control

Version	Date	Notes
v0.4 DRAFT	2026-08-29	Many streams, not one. New §1b : futures, crypto, forex and economic streams were part of the original vision, so the stream is a stream <i>of a kind</i> and each kind has its own shape rather than one shape stretched to cover all of them. The contract absorbs it — a part declares the kind it needs — which is the §2.1 mechanism with a buried assumption removed. Opens named and not answered: whether a part can read several kinds and whether that is one context or several, and that the playhead becomes the clock of a kind under replay. And the one consequence that cannot be deferred: each kind needs its own archive, capture is permanent-backward, and a stream worth having later should be captured before the product that wants it — the lesson the wing band, the ODTE limit and the VIX proxy each taught once.

v0.3 DRAFT	2026-08-29	The level the first two drafts missed. New §1a — one stream, many consumers: every tool derives from one common stream, the source is pluggable, the shape is fixed, and parts never know which one they read. Four consequences, including that replay is a property of the stream rather than a feature each product implements, and that a third source plugs in behind the same shape for free. New §2.2 — parts have faces, not one presentation: a desk face and an Analyzer overlay are the same product shown two ways, which is why owning GEX makes it appear in both; the Analyzer is settled as a surface, not a part. New §2.3 — capabilities as a second class: Time Machine is one entitlement across every surface, declared in a part's <i>needs</i>, priced on what it makes possible. New §4.2 — surfaces are unbounded, familiar-varied and unique, with Width Fit Rank as the case a template engine cannot produce. New §8a — Strategy Lab is the same paradigm with the arrow reversed, inheriting the contract and the consequence vocabulary.
v0.2 DRAFT	2026-08-29	The rudimentary build has a job. Retention of Observers whose trial ends (spec §0a), which changes what v1 must contain. Funnel diagram redrawn with two entrances — the trial path first, the public path later, both converging on one member. New §3.1 on what path A requires that path B does not: a desk that is not empty, a success test of <i>does a lapsed Observer return</i>, and an honest transition. New §4.1 — the gift is the first two parts, and building the environment around Verticals and Butterflies is a design advantage because every contract question has a real answer for them.
v0.1 DRAFT	2026-08-29	High-level design against the spec. Six objects; the part contract as the load-bearing declaration; four diagrams — member funnel, Runner execution environment, constellation and promotion gate, Factory pull. Temporary-part rules named as the third use of an existing pattern. Delivery paths shown to depend on the contract. Four unverified facts named as blocking a plan. No opens decided. Not BUILD AUTHORITY.

One-line law:

A part declares the stream it needs, what it produces and what shape it requires; the environment satisfies it or refuses; the constellation is complete or it is not staged; a human promotes it; everything a member can click is a thing that will work; the first desk anyone lands on already does something worth returning to; and every stream worth replaying was captured before anyone asked for it.